3.1.5 Numerical Precision

Integers and rational numbers in *Mathematica* are treated exactly. Real numbers, however, can usually only be treated approximately. The reason is that even to specify an arbitrary real number exactly can take an infinite number of digits. On a finite computer, therefore, one has to work with approximations to real numbers that involve only a finite number of digits.

In a symbolic system like *Mathematica*, however, you can nevertheless often represent real numbers exactly. For example, the symbol `Pi` is an exact representation of the mathematical constant π. However, when you ask for the numerical value of π using `N`, *Mathematica* can give you only a finite precision result.

`Precision[x]`	the number of decimal digits of precision in x
`Accuracy[x]`	the number of significant digits to the right of the decimal point in x

Precision and accuracy of real numbers.

The precision of an exact integer is infinite.

```
In[1]:= Precision[3]
Out[1]= Infinity
```

This gives the numerical value of π^{20} to 20 significant figures.

```
In[2]:= pi20 = N[Pi^20, 20]
Out[2]= 8769956796.0826994748
```

The precision is equal to the total number of decimal digits. *Mathematica* sometimes does computations to slightly more digits than you explicitly ask for.

```
In[3]:= Precision[pi20]
Out[3]= 25
```

The accuracy is the number of digits that appear to the right of the decimal point.

```
In[4]:= Accuracy[pi20]
Out[4]= 14
```

Whenever you enter a real number, *Mathematica* has to assign a definite precision to it. However, in the standard way of entering real numbers, you cannot explicitly specify the precision.

Mathematica always assumes that the digits you explicitly type in are accurate. The question is then whether you intend these digits to be followed by some number of zeroes, or not.

Mathematica distinguishes two kinds of approximate real numbers, and makes different assumptions about them. The first kind are "low precision numbers", that

Mathematica assumes are always accurate to some definite "machine precision". (As discussed in Section 3.1.7, the machine precision is usually 16 decimal digits, but may vary from one computer system to another.)

The second kind of real numbers are "high precision" ones, which *Mathematica* assumes are accurate only to the number of digits you explicitly give.

Low precision numbers (e.g `3.`): assumed accurate to a definite machine precision.

High precision numbers (e.g. `3.0000000000000000`): assumed accurate only to the number of digits you explicitly give.

Two types of approximate real numbers in *Mathematica*.

Here is a "high precision number".

```
In[5]:= 3.00000000000000000000000000
Out[5]= 3.
```

The precision is equal to the number of digits you explicitly gave.

```
In[6]:= Precision[ % ]
Out[6]= 30
```

`3.` is a "low precision number". Its precision is equal to the fixed "machine precision". *Mathematica* effectively adds zeroes to the right of the decimal point until this precision is reached.

```
In[7]:= Precision[ 3. ]
Out[7]= 16
```

Whenever you give just a few digits, *Mathematica* treats your numbers as "low precision", and assigns a definite machine precision to them.

```
In[8]:= Precision[ 3.00 ]
Out[8]= 16
```

In doing numerical computations, it is inevitable that you will sometimes end up with results that are less precise than you want. Particularly when you get numerical results that are very close to zero, you may well want to *assume* that the results should be exactly zero.

`Chop[`*expr*`]`	replace all approximate real numbers in *expr* with magnitude less than 10^{-10} by 0
`Chop[`*expr*`,` *dx*`]`	replace numbers with magnitude less than *dx* by 0

Removing numbers close to zero.

Web sample page from The *Mathematica* Book, First Edition, by Stephen Wolfram, published by Addison-Wesley Publishing Company (hardcover ISBN 0-201-19334-5; softcover ISBN 0-201-19330-2). To order *Mathematica* or this book contact Wolfram Research: info@wolfram.com; http://www.wolfram.com/; 1-800-441-6284.

Here is a 2×2 numerical matrix.

```
In[9]:= m = {{2., -3.}, {4., -7.}}

Out[9]= {{2., -3.}, {4., -7.}}
```

Multiplying the matrix by its inverse should give an identity matrix. Instead, there are some small off-diagonal terms.

```
In[10]:= Inverse[m] . m

                           -15                    -15
Out[10]= {{1., -7.10543 10   }, {2.66454 10   , 1.}}
```

You can use `Chop` to remove the off-diagonal terms.

```
In[11]:= Chop[%]

Out[11]= {{1., 0}, {0, 1.}}
```